

Assignment 5 Report Template

From Supervised Learning to Self-Supervised Learning using SimCLR

Student Name: Saad Ali

Roll Number: MSDS25066

GitHub Repository URL:

https://github.com/Hashimi321/DL_Assignment5_SimCLR_MSDS25066

1. Conceptual Warm-up:

Q1. In supervised classification, what information does the model learn from?

In supervised classification, the model learns from **labeled examples** — real data where someone has already said "*this is a cat*" or "*this is a dog*." The model keeps comparing what it predicts to the correct answer, and every time it's wrong, it adjusts itself to do better next time.

Q2. If CIFAR-10 labels are removed, do the images still contain useful visual structure?

Absolutely. Removing labels doesn't erase what's *in* the image. A cat still has fur, ears, and eyes — the pixels don't change just because the label is gone. The visual structure was always in the image itself; the label was just a name we attached to it.

Q3. If two augmented versions are created from the same original image, should their feature representations be similar or different?

They should be **similar**. No matter how you crop, flip, or recolor an image, it's still the same object underneath. Both versions share the same identity, so their feature representations should reflect that — different appearance, same meaning.

Q4. If two images come from different original images, should their feature representations be similar or different?

They should be **different**. Two separate images mean two separate things — a dog is not a car, no matter how you look at it. A good model should be able to tell them apart, producing clearly distinct representations for each.

Q5. Why might a model trained without labels still be useful for classification later?

Because it's not learning nothing — it's learning *how to see*. Even without labels, the model picks up on real visual patterns like shapes, textures, and edges. When you later introduce a few labels, the model already understands the visuals — it just needs to learn the names.

Q6. What is the difference between pretraining and fine-tuning?

Pretraining is where the model **learns the basics** — general patterns from lots of unlabeled data. Fine-tuning is where you **put it to work** — taking that already-knowledgeable model and teaching it a specific task with a small amount of labeled data. Think of it as education first, then specialization.

2. Training Settings:

Item	Value Used
Batch size	64
SimCLR epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Learning rate	3e-4 (0.0003)
Optimizer	Adam
Temperature	0.5
GPU/CPU used	Google Colab Tesla T4 GPU (training), CPU (evaluation)
Approximate training time	SimCLR: ~2.5 hours on Colab GPU, Fine-tuning: ~10 minutes

3. Supervised Baseline:

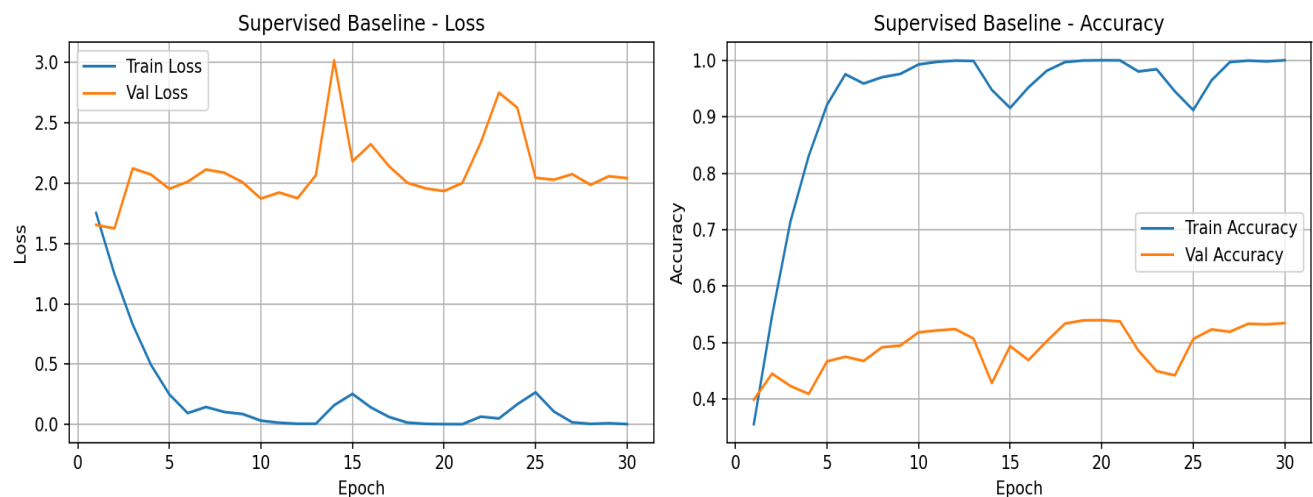
A ResNet-18 was trained **from scratch** on just 5,000 labeled images — only 10% of the full dataset. With limited data, cross-entropy loss, and Adam optimizer over 30 epochs, the model had to learn everything — features, patterns, structure — all from a small labeled pool alone.

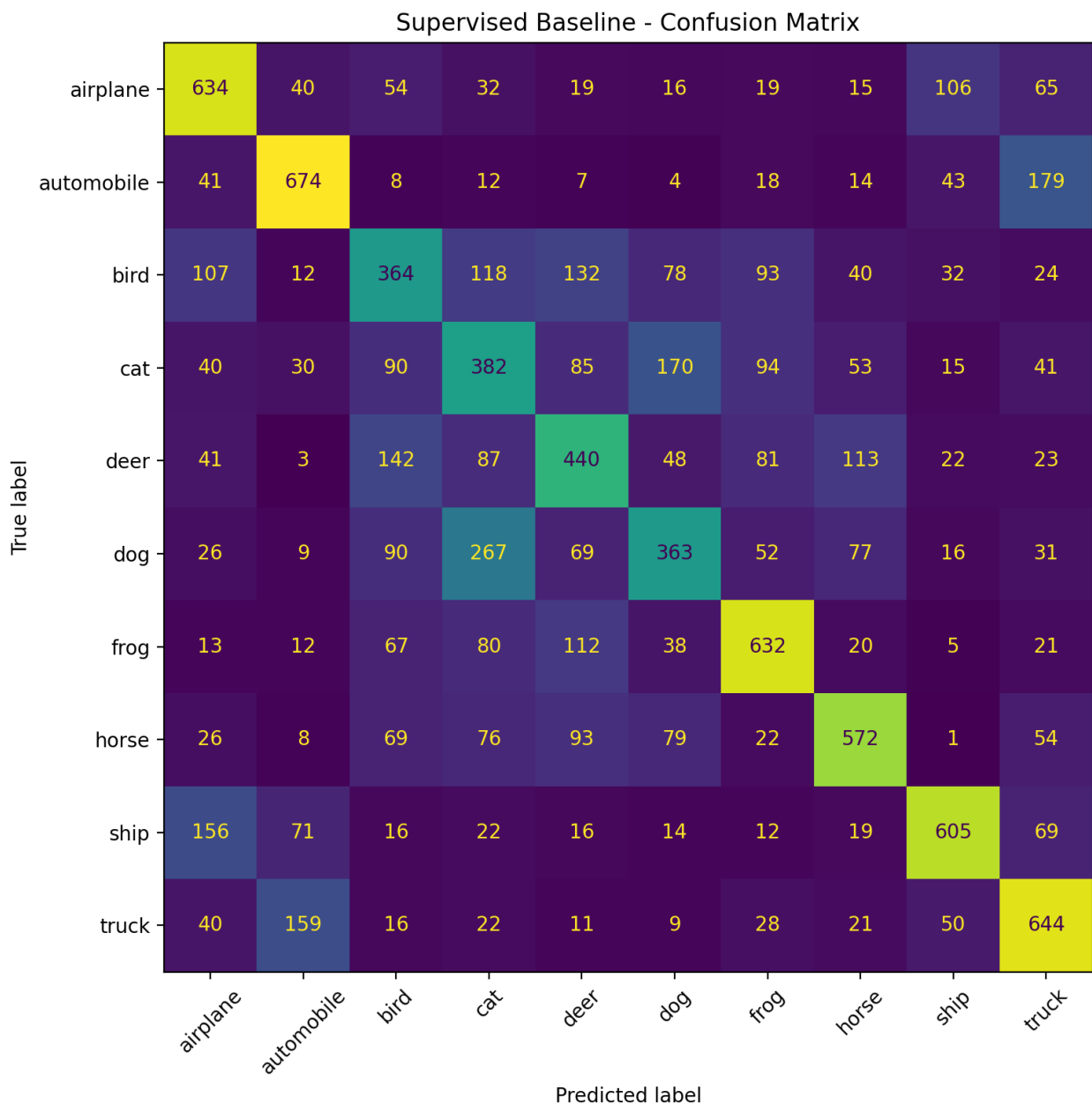
Results:

Metric	Value
Best Validation Accuracy	53.98%
Final Test Accuracy	53.10%
Most Confused Classes	Bird and Dog

Observations:

- The model **memorized** training data rather than learning — train loss dropped to near zero while validation loss kept climbing, a clear sign of overfitting.
- Validation loss **rose from 1.65 to 2.04** after early epochs, confirming the model stopped generalizing.
- It struggled most between **Birds and Dogs** — at 32×32 resolution, their shapes look too similar to tell apart reliably.
- The root cause was simple: **5,000 labeled samples isn't enough** for a model starting from scratch to learn meaningful patterns.



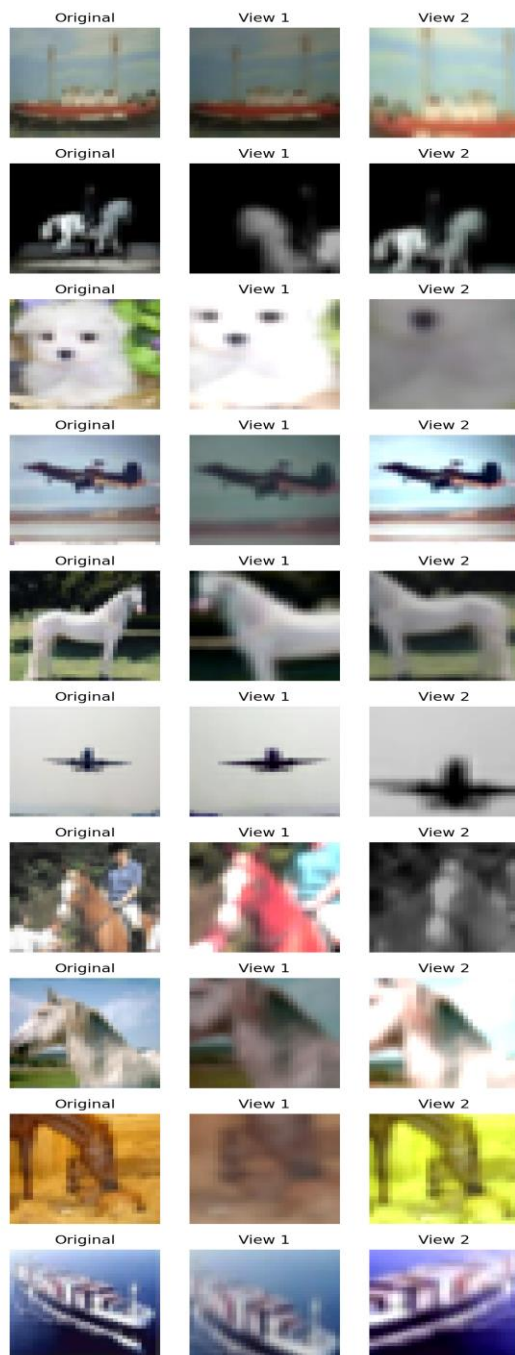


4. Augmentation Visualization:

SimCLR creates **two different views** of the same image by pushing it through six transforms:

- **RandomResizedCrop** — randomly crops and resizes back to 32×32, forcing the model to focus on different parts of the image.
- **RandomHorizontalFlip** — randomly mirrors the image, so orientation doesn't become a cue.

- **ColorJitter** — shifts brightness, contrast, saturation, and hue so the model doesn't rely on color alone.
- **RandomGrayscale** — occasionally strips all color out entirely, strengthening shape-based learning.
- **ToTensor** — converts the image into a format PyTorch can actually work with.
- **Normalize** — balances pixel values using CIFAR-10's mean and std, keeping inputs stable during training.



The TwoViewTransform simply runs the same augmentation pipeline twice on one image — producing two different-looking but same-meaning versions. SimCLR treats these as a positive pair because they're both the same object at heart.

- If augmentations are **too weak** — the two views look almost identical, so the model takes a shortcut and learns nothing real.
- If augmentations are **too strong** — the views look so different that the model can't connect them at all.

The goal is the **sweet spot** — different enough to challenge the model, similar enough to still make sense.

5. Feature Similarity Before Training

Before any training, a **randomly initialized encoder** was tested — images were passed through it and cosine similarity was measured between their feature vectors.

- Similar images showed **no meaningful similarity** — the encoder was just producing random noise as features.
- This confirmed that useful representations don't exist at the start — they have to be **learned**, not assumed.

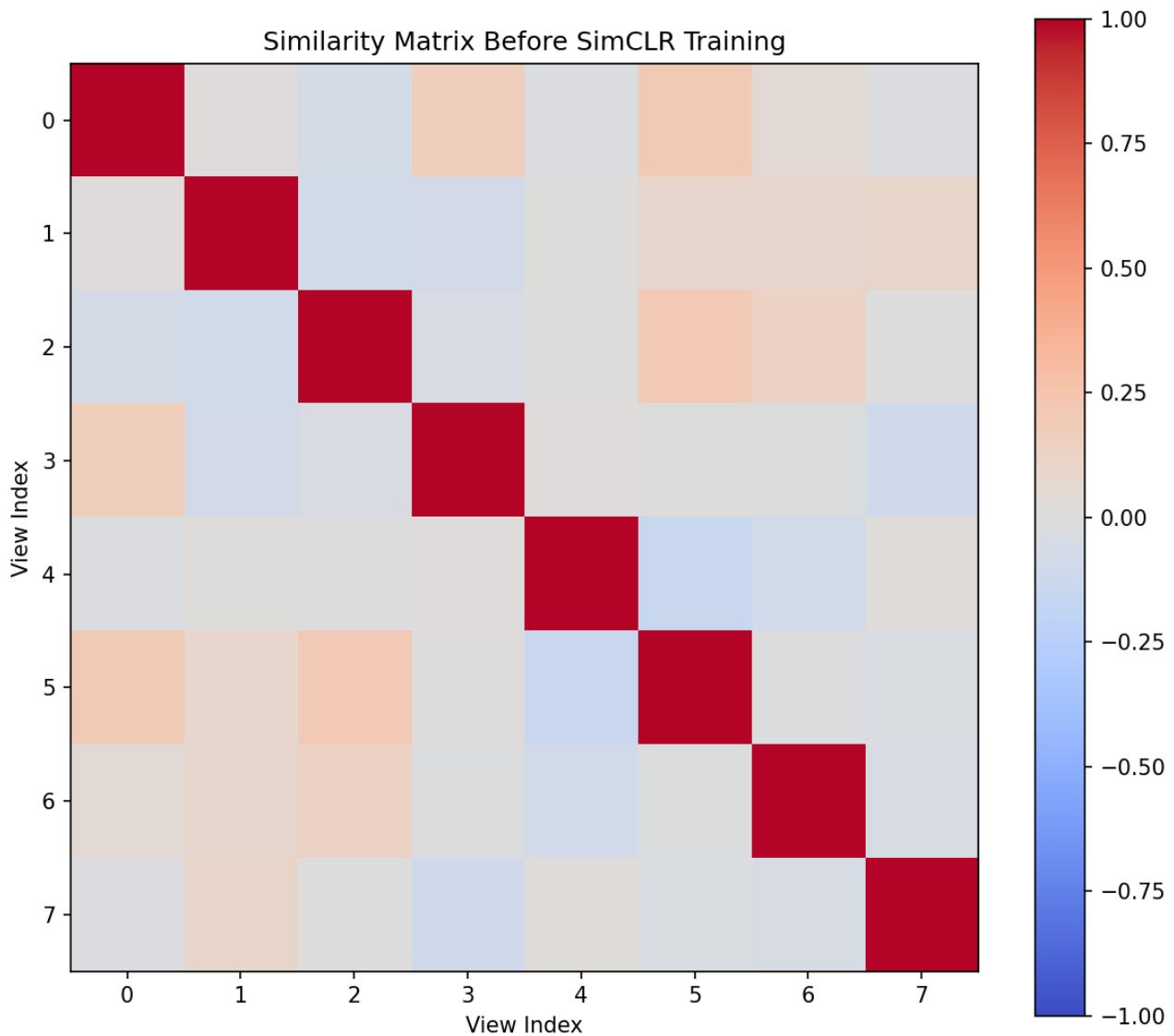
It set a clear baseline: without training, the encoder is blind.

Pair Type	Cosine Similarity
Same image (two augmented views)	0.9881
Different images	0.9845

Both similarity scores came out **close to 1.0** — nearly identical for both same and different image pairs. This reveals a clear problem:

- A **random encoder can't tell images apart** — it maps everything to roughly the same direction in feature space.
- Same pair or different pair — it **doesn't matter**, the scores look the same.

This proves that high cosine similarity means nothing without proper training — the encoder isn't seeing anything, it's just pointing everything the same way.



6. SimCLR Implementation:

Positive and Negative Pairs: For every image, SimCLR creates **two augmented views**, giving a batch of **2N total views**. With 4 images, that's **8 views total**:

- **Positive pair** — the two views of the *same* original image. They look different but mean the same thing, so their representations should be **pulled together**.
- **Negative pairs** — every view paired with views from *different* original images. These should be **pushed apart** in feature space.

So, for 4 images, each view has **1 positive pair** and **6 negative pairs** — the model is constantly learning to find the one match among many distractors.

Similarity Matrix: The similarity matrix is a $2N \times 2N$ grid comparing every view against every other:

- **Diagonal is ignored** — an image compared to itself always scores 1.0, which teaches the model nothing.
- **Positive pairs** sit in the **off-diagonal blocks** (top-right and bottom-left) — these are the two views of the same original image.
- **Everything else is a negative** — different original images that the model must learn to push apart.

The model's job is simple: in a sea of negatives, **find your match**.

NT-Xent Loss: NT-Xent (Normalized Temperature-scaled Cross Entropy) loss is the core learning signal in SimCLR. For each positive pair, it asks one question: *can you find your match among all the negatives?*

$$\text{Loss}(i, j) = -\log \left[\frac{\exp(\text{sim}(z_i, z_j) / \tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k) / \tau)} \right]$$

- **Numerator** — rewards high similarity between the two views of the same image.
- **Denominator** — compares that against similarity with all other views in the batch.
- **Temperature ($\tau = 0.5$)** — controls sharpness; lower temperature makes the model more confident and decisive about separating pairs.

In simple terms: **boost the positive, suppress the negatives**. The lower the loss, the better the model is at finding the right match in a crowd.

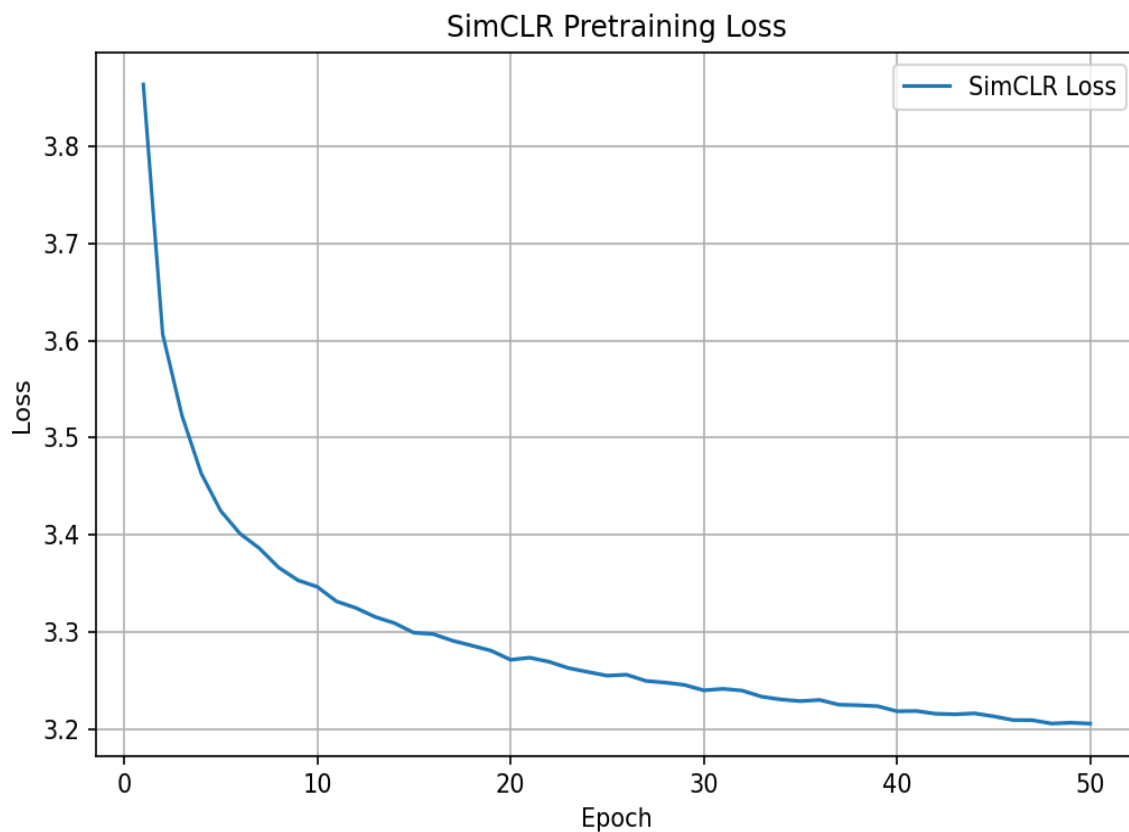
7. SimCLR Pretraining Results

Loss Curve

The loss dropped steadily from **3.86 to 3.20** over 50 epochs — not dramatic, but consistent and meaningful:

- The model was **genuinely learning** — not memorizing, just slowly understanding what makes two views the same.
- Each epoch, positive pairs got **pulled closer** and negatives got **pushed further apart** in feature space.

Slow, steady progress is actually a good sign in self-supervised learning — it means the model is building **real structure**, not taking shortcuts.



Feature Similarity Before vs After SimCLR

Pair Type	Before SimCLR	After SimCLR
Same image (two views)	0.9881	0.9245
Different images	0.9845	0.3221

After training, the difference became unmistakable — **0.92 for same-image pairs, 0.32 for different-image pairs**, a gap of 0.60:

- Before training, everything looked the same to the model. Now it can **clearly tell apart** what belongs together and what doesn't.
- And it did this **entirely without labels** — no one told it what was a cat or a car, it figured out the structure on its own.

A 0.60 gap is the model saying: *"I finally know what I'm looking at."*

8. Linear Probe Evaluation

Linear probing is a **fair test** of what the encoder actually learned:

- The encoder is **frozen** — its weights don't change at all.
- Only a simple **linear layer** ($512 \rightarrow 10$) is trained on top using labeled data.

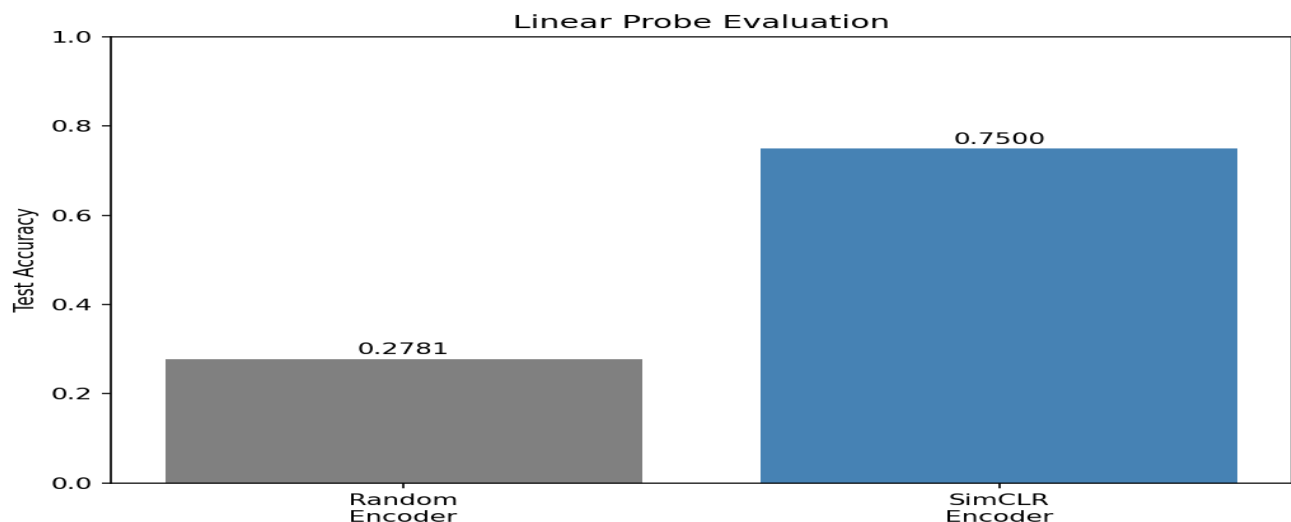
If the features are good, even a simple linear classifier will perform well. If accuracy is low, the encoder didn't learn meaningful structure. It's essentially asking: "**Are your features actually useful — or just noise?**"

Model	Encoder	Test Accuracy
Random Linear Probe	Random frozen encoder	27.81%
SimCLR Linear Probe	SimCLR frozen encoder	75.00%

The results speak for themselves:

- **SimCLR hit 75% accuracy** with a frozen encoder and no labels during pretraining — compared to just 27.81% from a random encoder, that's a **47% jump**.
- Even more striking — SimCLR's 75% **beat supervised training's 53.10%**, which actually used labeled data throughout.

The takeaway is clear: **learning to see without labels beats learning to memorize with limited ones**. SimCLR built genuine visual understanding — supervised training on 10% data just overfit and struggled.

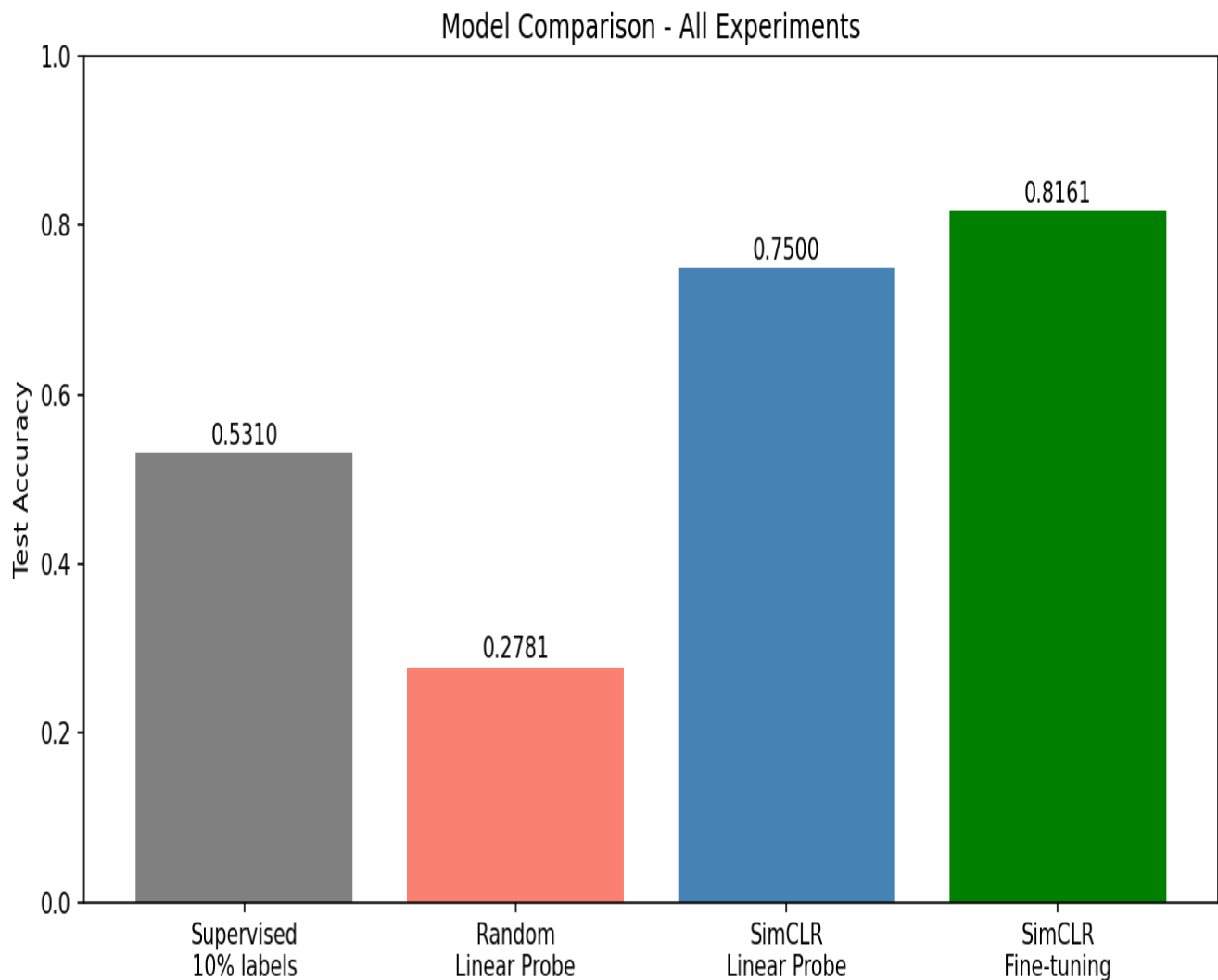


9. Fine-tuning Results:

Unlike linear probing where the encoder was frozen, fine-tuning **unlocks everything** — every weight in the model updates together:

- The pretrained encoder's learned features serve as a **strong starting point** rather than starting from scratch.
- Training end-to-end on the 10% labeled data for 20 epochs lets the model **adapt its features** specifically for CIFAR-10 classification.

Think of it as the difference between *testing* what the model knows versus *building on top of it*.



Experiment	Test Accuracy
Supervised ResNet-18 from scratch using 10% labels	53.10%
Random frozen encoder + linear classifier	27.81%
SimCLR frozen encoder + linear classifier	75.00%
SimCLR pretrained encoder + full fine-tuning	81.61%

Fine-tuning delivered the **best result across all experiments — 81.61% accuracy**:

- That's **28.51% better than supervised training** using the exact same labeled data.
- The pretrained features gave the model a massive head start — instead of learning from scratch, it was already halfway there.

The message is clear: **self-supervised pretraining on unlabeled data, followed by fine-tuning, is far more powerful than supervised training alone when labeled data is scarce**. Unlabeled data isn't useless — it just needed the right approach.

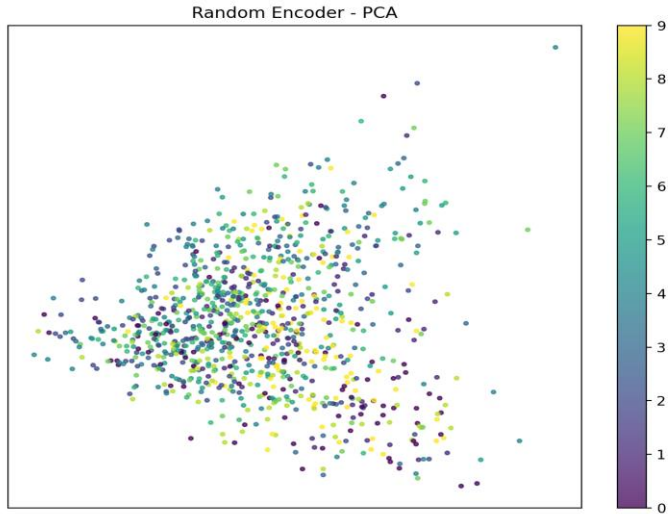
10. PCA Visualization:

512-dimensional features were extracted from 1000 validation images using each encoder and reduced to 2D using PCA (seed=2026). Labels were used only for coloring, not for training.

- **Labels were only used for coloring** — not fed into the model at any point, keeping the comparison honest.
- 1,000 validation images were passed through each encoder, and the resulting features were plotted to see how well the model **naturally grouped similar images together**.

It's a visual truth test — if the clusters look clean and separated, the encoder genuinely learned meaningful structure.

Random Encoder PCA:

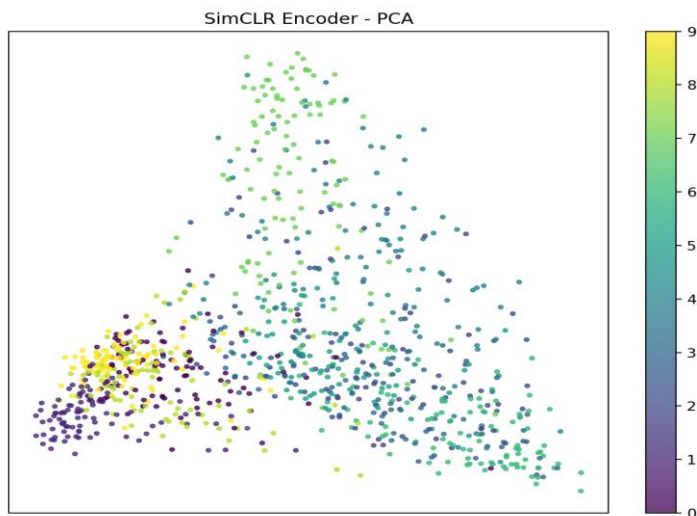


The random encoder's plot tells the whole story — **one big mixed blob**, no separation, no structure:

- Every class is tangled together, colors scattered randomly with no meaningful grouping.
- The encoder has **no idea** what it's looking at — it's just projecting images into random directions in feature space.

It's visual proof that **without training, there is no understanding** — just noise dressed up as numbers.

SimCLR Encoder PCA:

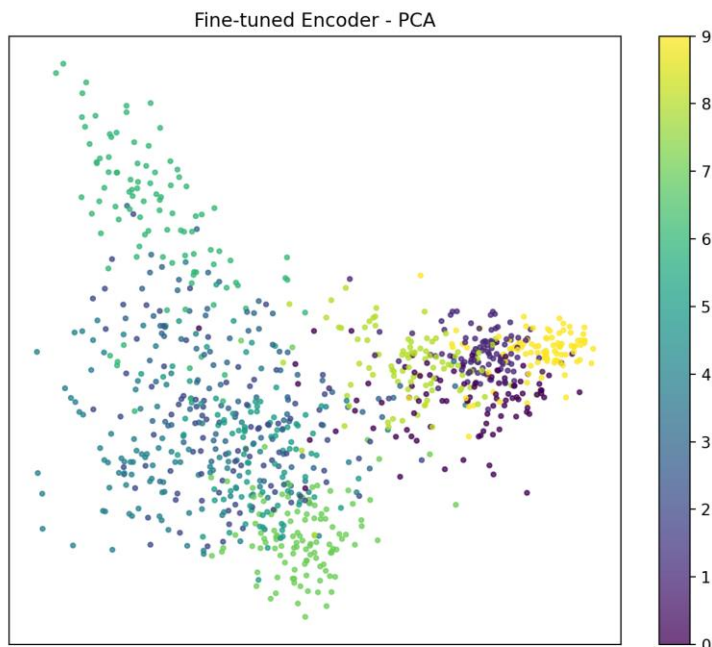


The SimCLR encoder's plot shows a **clear shift from chaos to structure** — a triangular/fan shape with real separation starting to form:

- **Trucks and airplanes** already have visible clusters — the model naturally grouped them without ever seeing a single label.
- The overall shape shows the model has learned that **some classes belong together and others don't**.

This structure didn't come from labels — it came from simply learning **what looks like what**. That's the power of self-supervised learning made visible.

Fine-tuned Encoder PCA:



The fine-tuned encoder's plot is the clearest of all — **two distinct groups, clean separation, real structure**:

- **Trucks are completely pulled apart** to the right — no overlap, no confusion.
- The overall layout is far more **organized and intentional** compared to SimCLR alone.

Fine-tuning took the rough structure SimCLR built and **sharpened it** — labels gave the model the final push to separate what it already suspected were different things.

Answers to Questions:

Q1: Do features from the random encoder show class-wise grouping?

- No — the random encoder shows pure chaos, every class randomly mixed with no structure whatsoever.

Q2: Do features from the SimCLR encoder show better grouping?

- Yes — SimCLR developed real, meaningful groupings entirely on its own, without ever seeing a label

Q3: Does fine-tuning improve class separation?

- Absolutely — fine-tuning delivered the sharpest separation, labels gave the model the final push it needed.

Q4: Which classes are still confused?

- Birds, Deer, and Cats still bleed into each other — visually too similar at 32×32 resolution to fully pull apart.

11. Final Comparison Table:

Model	Uses Labels During Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18, 10% labels	YES	NO	53.10%
Random encoder + linear probe	NO	YES	27.81%
SimCLR encoder + linear probe	NO	YES	75.00%
SimCLR encoder + fine-tuning	NO (pretraining) / YES (fine-tuning)	NO	81.61%

12. Discussion and Limitations:

What Worked:

- SimCLR learned real visual structure with zero labels — 75% linear probe accuracy crushed the 53.10% supervised baseline using the same data.
- Fine-tuning pushed it even further to **81.61%** — the best result, proving pretraining is a powerful head start.
- PCA visualizations told the story visually — clear progression from random noise → SimCLR structure → fine-tuned separation.
- Similarity matrix confirmed the model learned to tell same-image pairs from different ones — exactly what it was designed to do.

What Didn't Work as Expected:

- Pretraining loss only dropped from 3.86 to 3.20 — more epochs or a larger batch size could push it further.
- Birds, Deer, and Cats stayed confused even after fine-tuning — 32×32 resolution just doesn't give enough detail to separate visually similar classes.
- CPU training was practically unusable — SimCLR needs GPU power to run in any reasonable time.

Key Learnings:

- Self-supervised doesn't mean unsupervised — the supervision comes from the data itself, automatically.
- Augmentation quality is everything in SimCLR — too weak or too strong both break the learning signal.
- Good pretraining dramatically cuts down how much labeled data you actually need.
- NT-Xent loss does its job well — pulling same-image features together while pushing everything else apart.

AI USE LINK: <https://chatgpt.com/share/6a1f2549-a208-83aa-a62d-3052e2af59d9>